A project report on

ENGGBOT: A HIGHLY SCALABLE MULTI-MODAL CONVERSATIONAL AI PLATFORM

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

NAME OF THE STUDENT (REGISTRATION NO.)

VIT-AP UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

May, 2026

DECLARATION

I hereby declare that the thesis entitled "ENGGBOT: A HIGHLY SCALABLE MULTI-MODAL CONVERSATIONAL AI PLATFORM" submitted by Name of the Student (Registration No.) for the award of the degree of Bachelor of Technology in Computer Science and Engineering-core, VIT is a record of Bonafide work carried out by me under the supervision of Guide Name.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Amaravati
Date: 16-05-2026 **Signature of the Candidate**

CERTIFICATE

This is to certify that the Senior Design Project titled "ENGGBOT: A HIGHLY SCALABLE MULTI-MODAL CONVERSATIONAL AI PLATFORM" that is being submitted by Name of the Student (Registration No.) is in partial fulfilment of the requirements for the award of Bachelor of Technology, and is a record of bonafide work done under my guidance.

The contents of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

Guide Name
Internal Guide

Internal Examiner **External Examiner**

Approved by

DEAN
School Of Computer Science and Engineering

ABSTRACT

This project presents the development of a highly scalable, multi-modal conversational AI platform named ENGGBOT. The system architecture is designed to overcome common limitations in standard API wrappers by implementing a model-agnostic backend, dynamic prompt engineering, and a dual-pipeline speech recognition system. At the core of the natural language processing pipeline is an LLM multiplexing gateway powered by OpenRouter, which decouples the application from any single AI vendor. This allows for dynamic routing to cost-effective, open-source models such as glm-4.5-air while maintaining the flexibility to utilize proprietary models when complex reasoning is required. To improve the accuracy and logical consistency of model outputs, a custom Thinking Mode was engineered, utilizing Chain-of-Thought prompt injection to guide the model's reasoning processes prior to generation.

Furthermore, to ensure strict adherence to the application's persona and mitigate base-model hallucinations, a robust interception middleware was developed. This layer utilizes regex-based sanitization and intent-matching to dynamically strip out underlying provider artifacts such as OpenAI or Anthropic mentions and enforce a consistent, branded identity.

In addition to text generation, the platform integrates an enterprise-grade Speech-to-Text architecture. By implementing a dual-pipeline approach - utilizing NVIDIA Riva via gRPC for high-performance offline inference, and OpenAI's Whisper API as a cloud-based fallback - the system guarantees low latency and high resilience for audio processing tasks. The resulting application demonstrates a comprehensive, production-ready approach to AI integration, highlighting advanced techniques in prompt engineering, context management, and model orchestration.

The report also incorporates a Retrieval-Augmented Generation layer in which uploaded institutional and technical documents are processed through OCR, chunking, embedding generation, vector indexing, and hybrid retrieval. This extension allows ENGGBOT to combine model-agnostic conversational generation with grounded answers supported by repository content and multimodal document evidence.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Guide Name, School of Computer Science and Engineering, VIT-AP, for constant guidance, continual encouragement, understanding, and valuable suggestions throughout the completion of this project.

I would like to express my gratitude to the leadership of VIT-AP University and the School of Computer Science and Engineering for providing an environment to work in and for inspiring academic and technical growth during the course.

I also thank all teaching staff and members working as part of the university for their timely encouragement, which prompted the acquisition of the knowledge required to complete this work successfully. I would like to thank my parents for their support.

It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task. At last but not least, I express my gratitude and appreciation to all those who helped me directly or indirectly toward the successful completion of this project.

Place: Amaravati
Date: 16-05-2026

Name of the student

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS

AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
CoT	Chain-of-Thought
gRPC	Google Remote Procedure Call
LLM	Large Language Model
NLP	Natural Language Processing
OAuth	Open Authorization
RAG	Retrieval-Augmented Generation
SSE	Server-Sent Events
STT	Speech-to-Text
UI	User Interface
UX	User Experience
VQA	Visual Question Answering
RAM	Random Access Memory
SSD	Solid State Drive
GPU	Graphics Processing Unit

Chapter 1
Introduction

ENGGBOT is designed as a complete conversational AI platform rather than a thin wrapper around a single model endpoint. The project combines a polished web application, a model-agnostic natural language processing gateway, prompt control mechanisms, response middleware, and resilient speech processing. This combination allows the system to support text and voice interaction while preserving a consistent branded assistant identity.

The motivation for the project comes from the limitations observed in many basic chatbot implementations. Such systems often depend on one provider, expose raw model behavior to users, fail to control identity leakage, and treat speech input as an optional add-on. ENGGBOT addresses these issues through modular design and explicit orchestration layers.

The platform is particularly relevant for engineering education and technical assistance. Students require fast explanations, programming help, conceptual clarification, and interactive reasoning support. A multi-modal assistant improves this workflow by accepting typed prompts and speech input while returning structured responses through a modern user interface.

In the context of project overview, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how project overview contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

1.1 BACKGROUND

ENGGBOT is designed as a complete conversational AI platform rather than a thin wrapper around a single model endpoint. The project combines a polished web application, a model-agnostic natural language processing gateway, prompt control mechanisms, response middleware, and resilient speech processing. This combination allows the system to support text and voice interaction while preserving a consistent branded assistant identity.

The motivation for the project comes from the limitations observed in many basic chatbot implementations. Such systems often depend on one provider, expose raw model behavior to users, fail to control identity leakage, and treat speech input as an optional add-on. ENGGBOT addresses these issues through modular design and explicit orchestration layers.

The platform is particularly relevant for engineering education and technical assistance. Students require fast explanations, programming help, conceptual clarification, and interactive reasoning support. A multi-modal assistant improves this workflow by accepting typed prompts and speech input while returning structured responses through a modern user interface.

In the context of background, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how background contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

1.2 MOTIVATION

ENGGBOT is designed as a complete conversational AI platform rather than a thin wrapper around a single model endpoint. The project combines a polished web application, a model-agnostic natural language processing gateway, prompt control mechanisms, response middleware, and resilient speech processing. This combination allows the system to support text and voice interaction while preserving a consistent branded assistant identity.

The motivation for the project comes from the limitations observed in many basic chatbot implementations. Such systems often depend on one provider, expose raw model behavior to users, fail to control identity leakage, and treat speech input as an optional add-on. ENGGBOT addresses these issues through modular design and explicit orchestration layers.

The platform is particularly relevant for engineering education and technical assistance. Students require fast explanations, programming help, conceptual clarification, and interactive reasoning support. A multi-modal assistant improves this workflow by accepting typed prompts and speech input while returning structured responses through a modern user interface.

In the context of motivation, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how motivation contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

1.3 PROBLEM STATEMENT

ENGGBOT is designed as a complete conversational AI platform rather than a thin wrapper around a single model endpoint. The project combines a polished web application, a model-agnostic natural language processing gateway, prompt control mechanisms, response middleware, and resilient speech processing. This combination allows the system to support text and voice interaction while preserving a consistent branded assistant identity.

The motivation for the project comes from the limitations observed in many basic chatbot implementations. Such systems often depend on one provider, expose raw model behavior to users, fail to control identity leakage, and treat speech input as an optional add-on. ENGGBOT addresses these issues through modular design and explicit orchestration layers.

The platform is particularly relevant for engineering education and technical assistance. Students require fast explanations, programming help, conceptual clarification, and interactive reasoning support. A multi-modal assistant improves this workflow by accepting typed prompts and speech input while returning structured responses through a modern user interface.

In the context of problem statement, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how problem statement contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

1.4 OBJECTIVES

ENGGBOT is designed as a complete conversational AI platform rather than a thin wrapper around a single model endpoint. The project combines a polished web application, a model-agnostic natural language processing gateway, prompt control mechanisms, response middleware, and resilient speech processing. This combination allows the system to support text and voice interaction while preserving a consistent branded assistant identity.

The motivation for the project comes from the limitations observed in many basic chatbot implementations. Such systems often depend on one provider, expose raw model behavior to users, fail to control identity leakage, and treat speech input as an optional add-on. ENGGBOT addresses these issues through modular design and explicit orchestration layers.

The platform is particularly relevant for engineering education and technical assistance. Students require fast explanations, programming help, conceptual clarification, and interactive reasoning support. A multi-modal assistant improves this workflow by accepting typed prompts and speech input while returning structured responses through a modern user interface.

In the context of objectives, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how objectives contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

1.5 SCOPE OF WORK

ENGGBOT is designed as a complete conversational AI platform rather than a thin wrapper around a single model endpoint. The project combines a polished web application, a model-agnostic natural language processing gateway, prompt control mechanisms, response middleware, and resilient speech processing. This combination allows the system to support text and voice interaction while preserving a consistent branded assistant identity.

The motivation for the project comes from the limitations observed in many basic chatbot implementations. Such systems often depend on one provider, expose raw model behavior to users, fail to control identity leakage, and treat speech input as an optional add-on. ENGGBOT addresses these issues through modular design and explicit orchestration layers.

The platform is particularly relevant for engineering education and technical assistance. Students require fast explanations, programming help, conceptual clarification, and interactive reasoning support. A multi-modal assistant improves this workflow by accepting typed prompts and speech input while returning structured responses through a modern user interface.

In the context of scope of work, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how scope of work contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

1.6 EXPECTED OUTCOME

ENGGBOT is designed as a complete conversational AI platform rather than a thin wrapper around a single model endpoint. The project combines a polished web application, a model-agnostic natural language processing gateway, prompt control mechanisms, response middleware, and resilient speech processing. This combination allows the system to support text and voice interaction while preserving a consistent branded assistant identity.

The motivation for the project comes from the limitations observed in many basic chatbot implementations. Such systems often depend on one provider, expose raw model behavior to users, fail to control identity leakage, and treat speech input as an optional add-on. ENGGBOT addresses these issues through modular design and explicit orchestration layers.

The platform is particularly relevant for engineering education and technical assistance. Students require fast explanations, programming help, conceptual clarification, and interactive reasoning support. A multi-modal assistant improves this workflow by accepting typed prompts and speech input while returning structured responses through a modern user interface.

In the context of expected outcome, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how expected outcome contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

1.7 PROJECT CONSTRAINTS

ENGGBOT is designed as a complete conversational AI platform rather than a thin wrapper around a single model endpoint. The project combines a polished web application, a model-agnostic natural language processing gateway, prompt control mechanisms, response middleware, and resilient speech processing. This combination allows the system to support text and voice interaction while preserving a consistent branded assistant identity.

The motivation for the project comes from the limitations observed in many basic chatbot implementations. Such systems often depend on one provider, expose raw model behavior to users, fail to control identity leakage, and treat speech input as an optional add-on. ENGGBOT addresses these issues through modular design and explicit orchestration layers.

The platform is particularly relevant for engineering education and technical assistance. Students require fast explanations, programming help, conceptual clarification, and interactive reasoning support. A multi-modal assistant improves this workflow by accepting typed prompts and speech input while returning structured responses through a modern user interface.

In the context of project constraints, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how project constraints contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The requirements analysis also considers maintainability and operational clarity. Each feature is mapped to a technical responsibility so that future developers can locate the relevant layer without confusing user interface behavior with back-end orchestration.

This separation keeps the project realistic for deployment. It allows testing to be performed at the interface, middleware, gateway, and speech levels independently, which reduces the risk of hidden defects during integration.

1.8 REPORT ORGANIZATION

ENGGBOT is designed as a complete conversational AI platform rather than a thin wrapper around a single model endpoint. The project combines a polished web application, a model-agnostic natural language processing gateway, prompt control mechanisms, response middleware, and resilient speech processing. This combination allows the system to support text and voice interaction while preserving a consistent branded assistant identity.

The motivation for the project comes from the limitations observed in many basic chatbot implementations. Such systems often depend on one provider, expose raw model behavior to users, fail to control identity leakage, and treat speech input as an optional add-on. ENGGBOT addresses these issues through modular design and explicit orchestration layers.

The platform is particularly relevant for engineering education and technical assistance. Students require fast explanations, programming help, conceptual clarification, and interactive reasoning support. A multi-modal assistant improves this workflow by accepting typed prompts and speech input while returning structured responses through a modern user interface.

In the context of report organization, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how report organization contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The RAG-oriented extension of ENGGBOT treats institutional documents as an indexed knowledge base rather than as passive files. In this model, uploaded reports, manuals, policy documents, and engineering records are parsed, segmented, embedded, and retrieved before the language model generates an answer.

This approach improves grounding because the model is not expected to answer only from its pretrained knowledge. Instead, relevant document chunks are selected from the repository and supplied as context, allowing the final response to remain tied to available evidence.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

1.9 SUMMARY

ENGGBOT is designed as a complete conversational AI platform rather than a thin wrapper around a single model endpoint. The project combines a polished web application, a model-agnostic natural language processing gateway, prompt control mechanisms, response middleware, and resilient speech processing. This combination allows the system to support text and voice interaction while preserving a consistent branded assistant identity.

The motivation for the project comes from the limitations observed in many basic chatbot implementations. Such systems often depend on one provider, expose raw model behavior to users, fail to control identity leakage, and treat speech input as an optional add-on. ENGGBOT addresses these issues through modular design and explicit orchestration layers.

The platform is particularly relevant for engineering education and technical assistance. Students require fast explanations, programming help, conceptual clarification, and interactive reasoning support. A multi-modal assistant improves this workflow by accepting typed prompts and speech input while returning structured responses through a modern user interface.

In the context of summary, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how summary contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

Chapter 2
Literature And System Study

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of conversational ai evolution, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how conversational ai evolution contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

2.1 MODEL-AGNOSTIC GATEWAYS

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of model-agnostic gateways, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how model-agnostic gateways contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The model gateway separates application behavior from vendor-specific implementation details. This reduces dependency on a single provider and allows the system to choose a model according to availability, cost, response quality, or reasoning requirements.

Such abstraction is especially valuable for a final-year engineering project because the application can continue to evolve as new models become available. The same chat interface can support economical open-source models for routine tasks and stronger proprietary models for complex reasoning.

2.2 PROMPT ENGINEERING

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of prompt engineering, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how prompt engineering contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

2.3 CHAIN-OF-THOUGHT REASONING

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of chain-of-thought reasoning, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how chain-of-thought reasoning contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

2.4 RESPONSE GOVERNANCE

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of response governance, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how response governance contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The response governance layer is important because the underlying model may produce references to its provider, training background, or system behavior that are not aligned with the intended assistant identity. ENGGBOT therefore applies a controlled interception step before presenting the final answer.

This step improves user trust and gives the application a stable product identity. The same layer can also be extended later for safety checks, institution-specific answer policies, and audit logging without changing the user interface.

2.5 SPEECH RECOGNITION

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of speech recognition, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how speech recognition contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The speech layer is treated as an operational subsystem rather than as an optional accessory. Audio input must pass through format validation, transcription selection, response assembly, and fallback handling before it can be used reliably in a conversational workflow.

This design is useful in practical deployment because microphone quality, network availability, model latency, and server readiness can vary across sessions. By documenting these conditions, the project shows how speech support can remain usable even when one transcription service is unavailable.

2.6 RIVA-BASED ASR

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of riva-based asr, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how riva-based asr contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The source report describes a document-ingestion pipeline in which heterogeneous records such as PDFs, scanned files, Word documents, spreadsheets, and technical drawings are converted into searchable content. OCR extraction is applied to scanned material, while digital files are parsed directly before text cleaning and normalization.

After preprocessing, the content is divided into contextual chunks. Chunking keeps each retrieved unit small enough for efficient model use while preserving semantic continuity across adjacent sections.

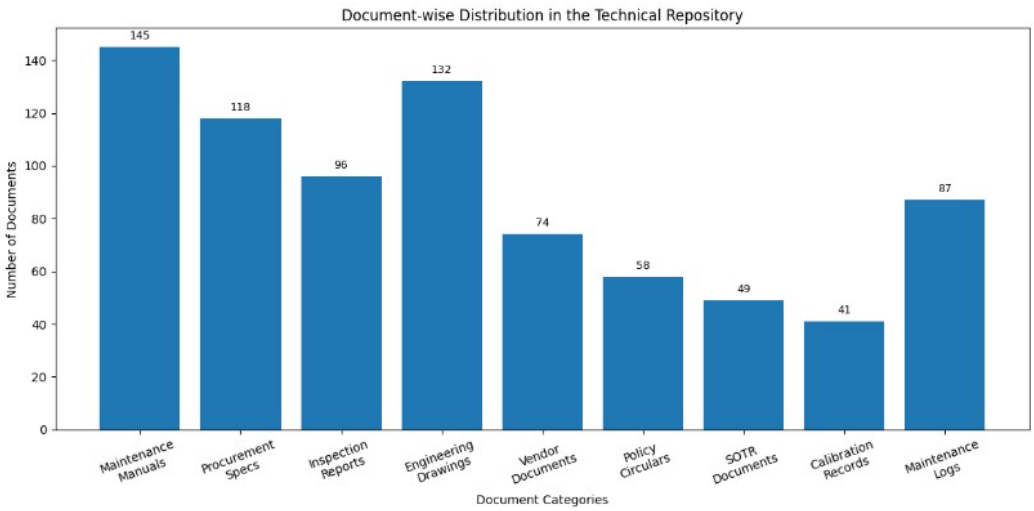

Fig 1: Document-wise distribution used for RAG preprocessing

The figure above presents the document-wise distribution used for rag preprocessing view used to connect the written design discussion with the implementation workflow. It gives a visual reference for how the RAG pipeline prepares content before the conversational model generates a response.

The figure also supports verification during review because it shows the document-processing or retrieval stage as a concrete system artifact rather than as a purely theoretical description.

The speech layer is treated as an operational subsystem rather than as an optional accessory. Audio input must pass through format validation, transcription selection, response assembly, and fallback handling before it can be used reliably in a conversational workflow.

This design is useful in practical deployment because microphone quality, network availability, model latency, and server readiness can vary across sessions. By documenting these conditions, the project shows how speech support can remain usable even when one transcription service is unavailable.

2.7 WHISPER-BASED ASR

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of whisper-based asr, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how whisper-based asr contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The speech layer is treated as an operational subsystem rather than as an optional accessory. Audio input must pass through format validation, transcription selection, response assembly, and fallback handling before it can be used reliably in a conversational workflow.

This design is useful in practical deployment because microphone quality, network availability, model latency, and server readiness can vary across sessions. By documenting these conditions, the project shows how speech support can remain usable even when one transcription service is unavailable.

2.8 FALLBACK ARCHITECTURES

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of fallback architectures, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how fallback architectures contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The design follows a layered approach because conversational AI systems involve multiple responsibilities that should not be mixed in a single module. User interaction, prompt formation, provider communication, response governance, and speech processing each remain separately understandable.

This organization also improves scalability. New capabilities, such as retrieval augmentation, additional speech engines, or analytics dashboards, can be attached to the appropriate layer without rewriting the complete application.

2.9 IDENTITY CONTROL

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of identity control, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how identity control contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The response governance layer is important because the underlying model may produce references to its provider, training background, or system behavior that are not aligned with the intended assistant identity. ENGGBOT therefore applies a controlled interception step before presenting the final answer.

This step improves user trust and gives the application a stable product identity. The same layer can also be extended later for safety checks, institution-specific answer policies, and audit logging without changing the user interface.

2.10 SCALABILITY PATTERNS

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of scalability patterns, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how scalability patterns contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

2.11 GAPS IDENTIFIED

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of gaps identified, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how gaps identified contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

2.12 SUMMARY

Area	Current Handling	Reason	Improvement
Model Access	OpenRouter gateway	Avoids vendor lock-in	Policy-based routing
Reasoning	Thinking Mode	Guides multi-step answers	Quality scoring
Identity	Response middleware	Prevents provider leakage	Larger test set
Speech	Riva and Whisper	Improves resilience	Automatic failover

A review of modern conversational AI systems shows that production readiness depends on more than the choice of base model. Practical systems require model routing, prompt templates, latency handling, fallback behavior, stream processing, observability, and output governance. ENGGBOT adopts these ideas through a gateway-oriented architecture.

Prompt engineering is a central element of the platform. The system does not rely only on a user's raw question. It enriches requests with controlled instructions, optional Thinking Mode guidance, and persona constraints. This improves coherence and allows the assistant to reason more explicitly when the task demands multi-step analysis.

Speech recognition is also studied as a reliability problem. A single speech provider may fail due to network conditions, credential limits, endpoint changes, or local device issues. Therefore, ENGGBOT uses a dual-pipeline approach in which high-performance Riva processing and Whisper-based fallback can support each other.

In the context of summary, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how summary contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The RAG framework uses embedding generation to convert text chunks and extracted annotations into dense vector representations. These embeddings allow the retrieval layer to identify conceptually related information even when the query and the source document use different wording.

Vector indexing is handled through a vector database such as Qdrant. Each

indexed item can store the embedding together with metadata including document type, section title, page reference, repository source, and engineering identifiers, making semantic retrieval and metadata filtering work together.

TABLE 1: Comparison between Traditional Search and Proposed RAG Syste

Parameter	Traditional System	Proposed RAG System
Retrieval Method	Exact keyword matching	Semantic and contextual retrieval
Search Capability	Keyword-based	Natural language understanding
Context Awareness	Low	High
Cross-Document Reasoning	Not supported	Supported
Engineering Drawing Retrieval	Limited	Multimodal retrieval
Semantic Understanding	Not available	Transformer embeddings
Citation Support	Not supported	Supported
SOTR Generation	Manual	Automated draft generation
Deployment	Traditional systems	On-premise AI framework
Security	Moderate	Air-gapped deployment

Fig 2: Comparison between traditional search and proposed RAG system

The figure above presents the comparison between traditional search and proposed rag system view used to connect the written design discussion with the implementation workflow. It gives a visual reference for how the RAG pipeline prepares content before the conversational model generates a response.

The figure also supports verification during review because it shows the document-processing or retrieval stage as a concrete system artifact rather than as a purely theoretical description.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

Chapter 3
Requirements And Analysis

The requirements analysis focuses on usability, scalability, correctness, latency, resilience, and maintainability. A conversational AI platform must feel responsive to the user, but it must also protect internal implementation details and remain flexible as AI providers change.

Functional requirements include authentication, chat interaction, model selection, thinking mode control, response streaming, speech input, conversation management, user profile handling, and consistent rendering of assistant responses. Non-functional requirements include availability, portability, security, and graceful degradation.

The system is feasible because the chosen stack separates responsibilities clearly. The user interface is implemented independently from model access, while middleware and speech services can evolve without replacing the entire application.

In the context of user requirements, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how user requirements contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

3.1 FUNCTIONAL REQUIREMENTS

Area	Current Handling	Reason	Improvement
Model Access	OpenRouter gateway	Avoids vendor lock-in	Policy-based routing
Reasoning	Thinking Mode	Guides multi-step answers	Quality scoring
Identity	Response middleware	Prevents provider leakage	Larger test set
Speech	Riva and Whisper	Improves resilience	Automatic failover

The requirements analysis focuses on usability, scalability, correctness, latency, resilience, and maintainability. A conversational AI platform must feel responsive to the user, but it must also protect internal implementation details and remain flexible as AI providers change.

Functional requirements include authentication, chat interaction, model selection, thinking mode control, response streaming, speech input, conversation management, user profile handling, and consistent rendering of assistant responses. Non-functional requirements include availability, portability, security, and graceful degradation.

The system is feasible because the chosen stack separates responsibilities clearly. The user interface is implemented independently from model access, while middleware and speech services can evolve without replacing the entire application.

In the context of functional requirements, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how functional requirements contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The requirements analysis also considers maintainability and operational clarity. Each feature is mapped to a technical responsibility so that future developers can locate the relevant layer without confusing user interface behavior with back-end orchestration.

This separation keeps the project realistic for deployment. It allows testing to be performed at the interface, middleware, gateway, and speech levels independently,

which reduces the risk of hidden defects during integration.

3.2 NON-FUNCTIONAL REQUIREMENTS

The requirements analysis focuses on usability, scalability, correctness, latency, resilience, and maintainability. A conversational AI platform must feel responsive to the user, but it must also protect internal implementation details and remain flexible as AI providers change.

Functional requirements include authentication, chat interaction, model selection, thinking mode control, response streaming, speech input, conversation management, user profile handling, and consistent rendering of assistant responses. Non-functional requirements include availability, portability, security, and graceful degradation.

The system is feasible because the chosen stack separates responsibilities clearly. The user interface is implemented independently from model access, while middleware and speech services can evolve without replacing the entire application.

In the context of non-functional requirements, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how non-functional requirements contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The requirements analysis also considers maintainability and operational clarity. Each feature is mapped to a technical responsibility so that future developers can locate the relevant layer without confusing user interface behavior with back-end orchestration.

This separation keeps the project realistic for deployment. It allows testing to be performed at the interface, middleware, gateway, and speech levels independently, which reduces the risk of hidden defects during integration.

3.3 SYSTEM CONSTRAINTS

The requirements analysis focuses on usability, scalability, correctness, latency, resilience, and maintainability. A conversational AI platform must feel responsive to the user, but it must also protect internal implementation details and remain flexible as AI providers change.

Functional requirements include authentication, chat interaction, model selection, thinking mode control, response streaming, speech input, conversation management, user profile handling, and consistent rendering of assistant responses. Non-functional requirements include availability, portability, security, and graceful degradation.

The system is feasible because the chosen stack separates responsibilities clearly. The user interface is implemented independently from model access, while middleware and speech services can evolve without replacing the entire application.

In the context of system constraints, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how system constraints contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The requirements analysis also considers maintainability and operational clarity. Each feature is mapped to a technical responsibility so that future developers can locate the relevant layer without confusing user interface behavior with back-end orchestration.

This separation keeps the project realistic for deployment. It allows testing to be performed at the interface, middleware, gateway, and speech levels independently, which reduces the risk of hidden defects during integration.

3.4 SECURITY REQUIREMENTS

The requirements analysis focuses on usability, scalability, correctness, latency, resilience, and maintainability. A conversational AI platform must feel responsive to the user, but it must also protect internal implementation details and remain flexible as AI providers change.

Functional requirements include authentication, chat interaction, model selection, thinking mode control, response streaming, speech input, conversation management, user profile handling, and consistent rendering of assistant responses. Non-functional requirements include availability, portability, security, and graceful degradation.

The system is feasible because the chosen stack separates responsibilities clearly. The user interface is implemented independently from model access, while middleware and speech services can evolve without replacing the entire application.

In the context of security requirements, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how security requirements contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The requirements analysis also considers maintainability and operational clarity. Each feature is mapped to a technical responsibility so that future developers can locate the relevant layer without confusing user interface behavior with back-end orchestration.

This separation keeps the project realistic for deployment. It allows testing to be performed at the interface, middleware, gateway, and speech levels independently, which reduces the risk of hidden defects during integration.

3.5 PERFORMANCE REQUIREMENTS

The requirements analysis focuses on usability, scalability, correctness, latency, resilience, and maintainability. A conversational AI platform must feel responsive to the user, but it must also protect internal implementation details and remain flexible as AI providers change.

Functional requirements include authentication, chat interaction, model selection, thinking mode control, response streaming, speech input, conversation management, user profile handling, and consistent rendering of assistant responses. Non-functional requirements include availability, portability, security, and graceful degradation.

The system is feasible because the chosen stack separates responsibilities clearly. The user interface is implemented independently from model access, while middleware and speech services can evolve without replacing the entire application.

In the context of performance requirements, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how performance requirements contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The requirements analysis also considers maintainability and operational clarity. Each feature is mapped to a technical responsibility so that future developers can locate the relevant layer without confusing user interface behavior with back-end orchestration.

This separation keeps the project realistic for deployment. It allows testing to be performed at the interface, middleware, gateway, and speech levels independently, which reduces the risk of hidden defects during integration.

3.6 RELIABILITY REQUIREMENTS

The requirements analysis focuses on usability, scalability, correctness, latency, resilience, and maintainability. A conversational AI platform must feel responsive to the user, but it must also protect internal implementation details and remain flexible as AI providers change.

Functional requirements include authentication, chat interaction, model selection, thinking mode control, response streaming, speech input, conversation management, user profile handling, and consistent rendering of assistant responses. Non-functional requirements include availability, portability, security, and graceful degradation.

The system is feasible because the chosen stack separates responsibilities clearly. The user interface is implemented independently from model access, while middleware and speech services can evolve without replacing the entire application.

In the context of reliability requirements, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how reliability requirements contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The requirements analysis also considers maintainability and operational clarity. Each feature is mapped to a technical responsibility so that future developers can locate the relevant layer without confusing user interface behavior with back-end orchestration.

This separation keeps the project realistic for deployment. It allows testing to be performed at the interface, middleware, gateway, and speech levels independently, which reduces the risk of hidden defects during integration.

3.7 FEASIBILITY STUDY

The requirements analysis focuses on usability, scalability, correctness, latency, resilience, and maintainability. A conversational AI platform must feel responsive to the user, but it must also protect internal implementation details and remain flexible as AI providers change.

Functional requirements include authentication, chat interaction, model selection, thinking mode control, response streaming, speech input, conversation management, user profile handling, and consistent rendering of assistant responses. Non-functional requirements include availability, portability, security, and graceful degradation.

The system is feasible because the chosen stack separates responsibilities clearly. The user interface is implemented independently from model access, while middleware and speech services can evolve without replacing the entire application.

In the context of feasibility study, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how feasibility study contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

Hybrid retrieval improves the basic RAG pipeline by combining semantic vector search with exact keyword retrieval. This is important in technical domains because semantic similarity is useful for conceptual questions, while exact matching is still needed for specification IDs, procurement references, drawing numbers, and equipment codes.

The source report also describes multimodal retrieval for engineering drawings. OCR extracts visible annotations, VQA-style descriptions provide semantic interpretation of diagrams, and CLIP-style visual embeddings support image similarity search across related schematics.

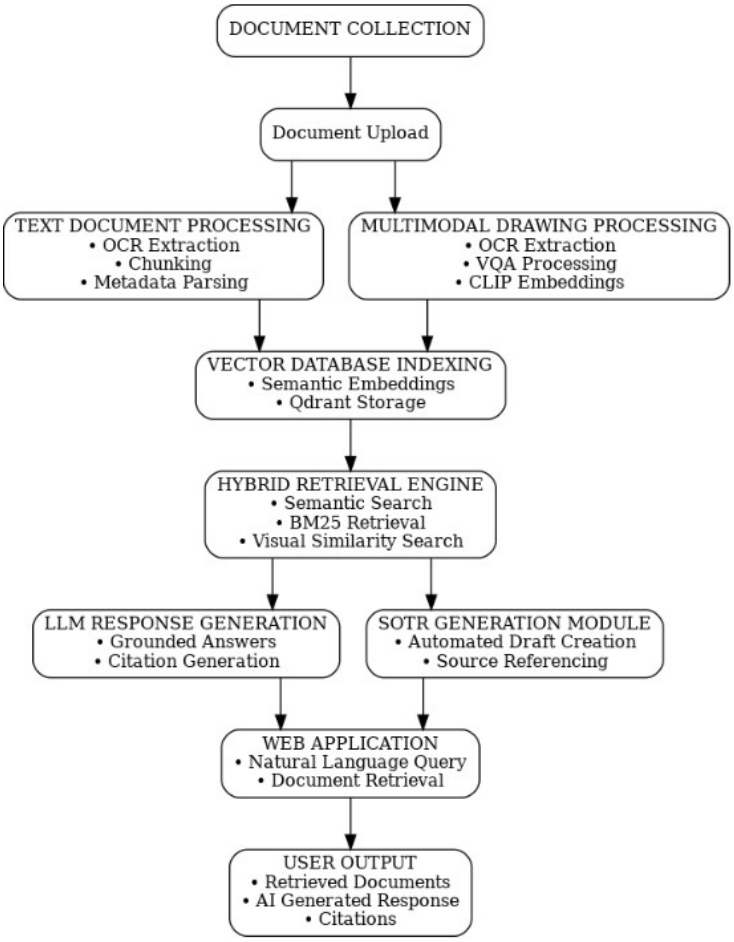

Fig 3: RAG document-processing and retrieval workflow

The figure above presents the rag document-processing and retrieval workflow view used to connect the written design discussion with the implementation workflow. It gives a visual reference for how the RAG pipeline prepares content before the conversational model generates a response.

The figure also supports verification during review because it shows the document-processing or retrieval stage as a concrete system artifact rather than as a purely theoretical description.

The requirements analysis also considers maintainability and operational clarity. Each feature is mapped to a technical responsibility so that future developers can locate the relevant layer without confusing user interface behavior with back-end orchestration.

This separation keeps the project realistic for deployment. It allows testing to be performed at the interface, middleware, gateway, and speech levels independently, which reduces the risk of hidden defects during integration.

3.8 TECHNOLOGY STACK

The requirements analysis focuses on usability, scalability, correctness, latency, resilience, and maintainability. A conversational AI platform must feel responsive to the user, but it must also protect internal implementation details and remain flexible as AI providers change.

Functional requirements include authentication, chat interaction, model selection, thinking mode control, response streaming, speech input, conversation management, user profile handling, and consistent rendering of assistant responses. Non-functional requirements include availability, portability, security, and graceful degradation.

The system is feasible because the chosen stack separates responsibilities clearly. The user interface is implemented independently from model access, while middleware and speech services can evolve without replacing the entire application.

In the context of technology stack, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how technology stack contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

3.9 SUMMARY

The requirements analysis focuses on usability, scalability, correctness, latency, resilience, and maintainability. A conversational AI platform must feel responsive to the user, but it must also protect internal implementation details and remain flexible as AI providers change.

Functional requirements include authentication, chat interaction, model selection, thinking mode control, response streaming, speech input, conversation management, user profile handling, and consistent rendering of assistant responses. Non-functional requirements include availability, portability, security, and graceful degradation.

The system is feasible because the chosen stack separates responsibilities clearly. The user interface is implemented independently from model access, while middleware and speech services can evolve without replacing the entire application.

In the context of summary, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how summary contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

Chapter 4
System Design

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of architectural overview, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how architectural overview contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

4.1 LAYERED DESIGN

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of layered design, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how layered design contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The design follows a layered approach because conversational AI systems involve multiple responsibilities that should not be mixed in a single module. User interaction, prompt formation, provider communication, response governance, and speech processing each remain separately understandable.

This organization also improves scalability. New capabilities, such as retrieval augmentation, additional speech engines, or analytics dashboards, can be attached to the appropriate layer without rewriting the complete application.

4.2 CLIENT LAYER

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of client layer, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how client layer contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The design follows a layered approach because conversational AI systems involve multiple responsibilities that should not be mixed in a single module. User interaction, prompt formation, provider communication, response governance, and speech processing each remain separately understandable.

This organization also improves scalability. New capabilities, such as retrieval augmentation, additional speech engines, or analytics dashboards, can be attached to the appropriate layer without rewriting the complete application.

4.3 CONTEXT LAYER

Area	Current Handling	Reason	Improvement
Model Access	OpenRouter gateway	Avoids vendor lock-in	Policy-based routing
Reasoning	Thinking Mode	Guides multi-step answers	Quality scoring
Identity	Response middleware	Prevents provider leakage	Larger test set
Speech	Riva and Whisper	Improves resilience	Automatic failover

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of context layer, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how context layer contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In an ENGGBOT deployment, RAG can be placed beside the existing OpenRouter model gateway. The gateway continues to route prompts to the selected model, while the retrieval layer prepares grounded context from institutional documents before the prompt is sent.

This separation keeps the application model-agnostic while improving factual

reliability. The conversational interface can remain the same, but answers become more useful because they are supported by repository content, metadata, and retrieved evidence.

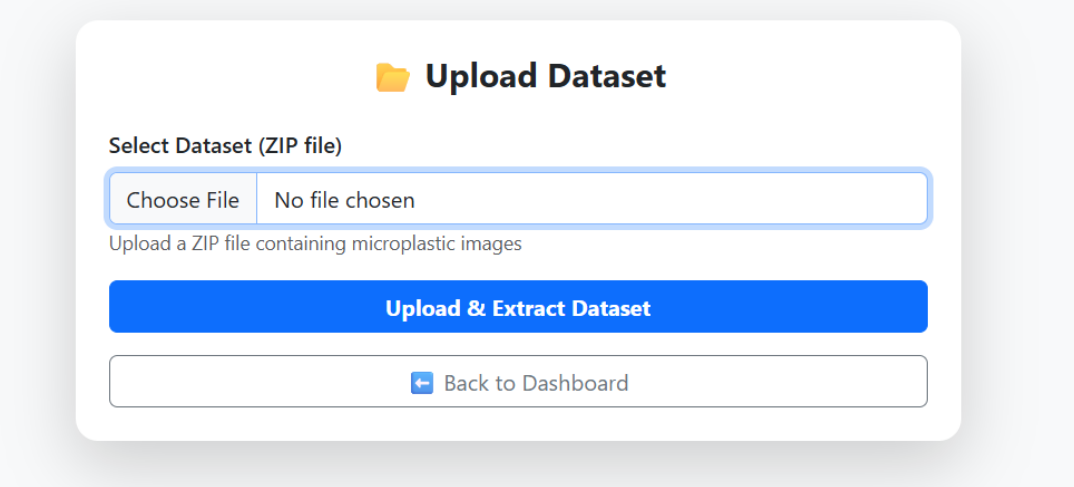

Fig 4: Dataset upload interface for repository ingestion

The figure above presents the dataset upload interface for repository ingestion view used to connect the written design discussion with the implementation workflow. It gives a visual reference for how the RAG pipeline prepares content before the conversational model generates a response.

The figure also supports verification during review because it shows the document-processing or retrieval stage as a concrete system artifact rather than as a purely theoretical description.

The design follows a layered approach because conversational AI systems involve multiple responsibilities that should not be mixed in a single module. User interaction, prompt formation, provider communication, response governance, and speech processing each remain separately understandable.

This organization also improves scalability. New capabilities, such as retrieval augmentation, additional speech engines, or analytics dashboards, can be attached to the appropriate layer without rewriting the complete application.

4.4 LLM GATEWAY

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of llm gateway, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how llm gateway contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The model gateway separates application behavior from vendor-specific implementation details. This reduces dependency on a single provider and allows the system to choose a model according to availability, cost, response quality, or reasoning requirements.

Such abstraction is especially valuable for a final-year engineering project because the application can continue to evolve as new models become available. The same chat interface can support economical open-source models for routine tasks and stronger proprietary models for complex reasoning.

4.5 MODEL ROUTING

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of model routing, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how model routing contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The model gateway separates application behavior from vendor-specific implementation details. This reduces dependency on a single provider and allows the system to choose a model according to availability, cost, response quality, or reasoning requirements.

Such abstraction is especially valuable for a final-year engineering project because the application can continue to evolve as new models become available. The same chat interface can support economical open-source models for routine tasks and stronger proprietary models for complex reasoning.

4.6 PROMPT BUILDER

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of prompt builder, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how prompt builder contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The interface is designed to make the assistant usable during repeated academic and engineering tasks. The chat surface must preserve conversation flow, show responses clearly, and expose speech or thinking controls without distracting the user from the main interaction.

A clean interface also supports evaluation. When the input, generated answer, and interaction mode are visible, it becomes easier to compare responses across models and to identify whether a defect belongs to the front end, middleware, or model gateway.

4.7 MIDDLEWARE DESIGN

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of middleware design, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how middleware design contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

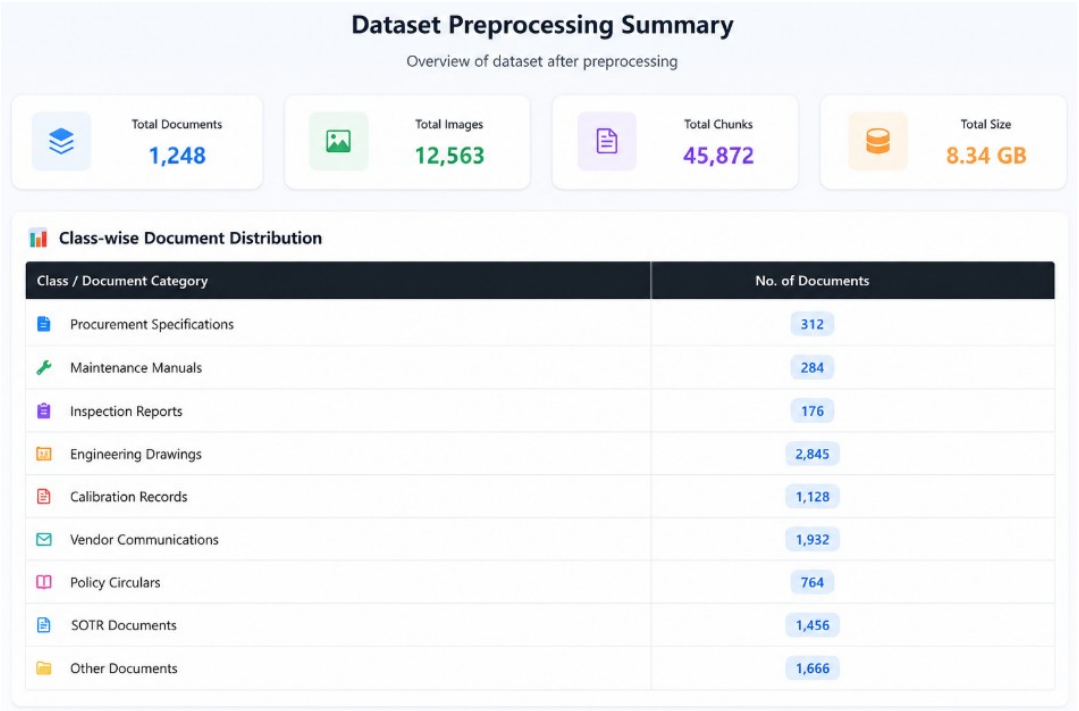

Fig 5: Dataset preprocessing summary dashboard

The figure above presents the dataset preprocessing summary dashboard view used to connect the written design discussion with the implementation workflow. It gives a visual reference for how the RAG pipeline prepares content before the conversational model generates a response.

The figure also supports verification during review because it shows the document-processing or retrieval stage as a concrete system artifact rather than as a purely theoretical description.

The response governance layer is important because the underlying model may produce references to its provider, training background, or system behavior that are not aligned with the intended assistant identity. ENGGBOT therefore applies a controlled interception step before presenting the final answer.

This step improves user trust and gives the application a stable product identity. The same layer can also be extended later for safety checks, institution-specific answer policies, and audit logging without changing the user interface.

4.8 IDENTITY INTERCEPTION

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of identity interception, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how identity interception contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The response governance layer is important because the underlying model may produce references to its provider, training background, or system behavior that are not aligned with the intended assistant identity. ENGGBOT therefore applies a controlled interception step before presenting the final answer.

This step improves user trust and gives the application a stable product identity. The same layer can also be extended later for safety checks, institution-specific answer policies, and audit logging without changing the user interface.

4.9 RESPONSE SANITIZATION

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of response sanitization, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how response sanitization contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The response governance layer is important because the underlying model may produce references to its provider, training background, or system behavior that are not aligned with the intended assistant identity. ENGGBOT therefore applies a controlled interception step before presenting the final answer.

This step improves user trust and gives the application a stable product identity. The same layer can also be extended later for safety checks, institution-specific answer policies, and audit logging without changing the user interface.

4.10 SPEECH ARCHITECTURE

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of speech architecture, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how speech architecture contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The speech layer is treated as an operational subsystem rather than as an optional accessory. Audio input must pass through format validation, transcription selection, response assembly, and fallback handling before it can be used reliably in a conversational workflow.

This design is useful in practical deployment because microphone quality, network availability, model latency, and server readiness can vary across sessions. By documenting these conditions, the project shows how speech support can remain usable even when one transcription service is unavailable.

4.11 FALLBACK STRATEGY

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of fallback strategy, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how fallback strategy contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

4.12 DATA DESIGN

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of data design, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how data design contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The design follows a layered approach because conversational AI systems involve multiple responsibilities that should not be mixed in a single module. User interaction, prompt formation, provider communication, response governance, and speech processing each remain separately understandable.

This organization also improves scalability. New capabilities, such as retrieval augmentation, additional speech engines, or analytics dashboards, can be attached to the appropriate layer without rewriting the complete application.

4.13 DEPLOYMENT DESIGN

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of deployment design, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how deployment design contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The design follows a layered approach because conversational AI systems involve multiple responsibilities that should not be mixed in a single module. User interaction, prompt formation, provider communication, response governance, and speech processing each remain separately understandable.

This organization also improves scalability. New capabilities, such as retrieval augmentation, additional speech engines, or analytics dashboards, can be attached to the appropriate layer without rewriting the complete application.

4.14 SUMMARY

The system design follows a layered architecture. The client layer manages interaction, layout, message rendering, and mode toggles. The context layer maintains chat state, current model choices, conversation identity, and user preferences. The gateway layer communicates with OpenRouter to decouple the application from any single model provider.

The middleware layer enforces response governance. It detects identity-related prompts, sanitizes provider artifacts, and replaces unwanted base-model references with the ENGGBOT identity. This design is important because a multiplexed model backend may otherwise expose inconsistent provider names or internal model details.

The speech layer handles audio input through independent processing paths. Riva supports high-performance speech recognition using gRPC-based workflows, while Whisper-based processing provides cloud fallback and broader compatibility. This improves resilience for real-world usage.

In the context of summary, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how summary contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

Chapter 5
Implementation Methodology

The implementation is divided across the main application, AI user interface, API routes, speech modules, and data services. The frontend provides an animated landing experience and a feature-rich chat interface. The backend provides authentication, model interaction, transcription endpoints, and persistence support.

OpenRouter integration gives ENGGBOT a model-agnostic foundation. The application can route requests to cost-effective open-source models such as glm-4.5-air while preserving the possibility of using more capable proprietary models for complex reasoning. This avoids vendor lock-in and improves long-term adaptability.

Thinking Mode is implemented as a request-time prompt modification strategy. When enabled, the system appends reasoning-oriented guidance so that the model structures its answer more carefully. This does not change the base model, but it changes the inference context in a predictable way.

In the context of frontend implementation, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how frontend implementation contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

5.1 CHAT INTERFACE

The implementation is divided across the main application, AI user interface, API routes, speech modules, and data services. The frontend provides an animated landing experience and a feature-rich chat interface. The backend provides authentication, model interaction, transcription endpoints, and persistence support.

OpenRouter integration gives ENGGBOT a model-agnostic foundation. The application can route requests to cost-effective open-source models such as glm-4.5-air while preserving the possibility of using more capable proprietary models for complex reasoning. This avoids vendor lock-in and improves long-term adaptability.

Thinking Mode is implemented as a request-time prompt modification strategy. When enabled, the system appends reasoning-oriented guidance so that the model structures its answer more carefully. This does not change the base model, but it changes the inference context in a predictable way.

In the context of chat interface, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how chat interface contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The interface is designed to make the assistant usable during repeated academic and engineering tasks. The chat surface must preserve conversation flow, show responses clearly, and expose speech or thinking controls without distracting the user from the main interaction.

A clean interface also supports evaluation. When the input, generated answer, and interaction mode are visible, it becomes easier to compare responses across models and to identify whether a defect belongs to the front end, middleware, or model gateway.

5.2 THINKING MODE

The implementation is divided across the main application, AI user interface, API routes, speech modules, and data services. The frontend provides an animated landing experience and a feature-rich chat interface. The backend provides authentication, model interaction, transcription endpoints, and persistence support.

OpenRouter integration gives ENGGBOT a model-agnostic foundation. The application can route requests to cost-effective open-source models such as glm-4.5-air while preserving the possibility of using more capable proprietary models for complex reasoning. This avoids vendor lock-in and improves long-term adaptability.

Thinking Mode is implemented as a request-time prompt modification strategy. When enabled, the system appends reasoning-oriented guidance so that the model structures its answer more carefully. This does not change the base model, but it changes the inference context in a predictable way.

In the context of thinking mode, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how thinking mode contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

In the context of The System, this section contributes to the overall understanding of ENGGBOT as an integrated engineering system. The discussion connects user needs, software architecture, model orchestration, and speech support into a single implementation narrative.

The section also records the design reasoning behind the selected approach. This is necessary in a final report because the value of the project is not limited to the working interface; it also lies in the engineering choices that make the platform scalable, maintainable, and extensible.

5.3 OPENROUTER INTEGRATION

The implementation is divided across the main application, AI user interface, API routes, speech modules, and data services. The frontend provides an animated landing experience and a feature-rich chat interface. The backend provides authentication, model interaction, transcription endpoints, and persistence support.

OpenRouter integration gives ENGGBOT a model-agnostic foundation. The application can route requests to cost-effective open-source models such as glm-4.5-air while preserving the possibility of using more capable proprietary models for complex reasoning. This avoids vendor lock-in and improves long-term adaptability.

Thinking Mode is implemented as a request-time prompt modification strategy. When enabled, the system appends reasoning-oriented guidance so that the model structures its answer more carefully. This does not change the base model, but it changes the inference context in a predictable way.

In the context of openrouter integration, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how openrouter integration contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The model gateway separates application behavior from vendor-specific implementation details. This reduces dependency on a single provider and allows the system to choose a model according to availability, cost, response quality, or reasoning requirements.

Such abstraction is especially valuable for a final-year engineering project because the application can continue to evolve as new models become available. The same chat interface can support economical open-source models for routine tasks and stronger proprietary models for complex reasoning.

5.4 STREAMING RESPONSE FLOW

The implementation is divided across the main application, AI user interface, API routes, speech modules, and data services. The frontend provides an animated landing experience and a feature-rich chat interface. The backend provides authentication, model interaction, transcription endpoints, and persistence support.

OpenRouter integration gives ENGGBOT a model-agnostic foundation. The application can route requests to cost-effective open-source models such as glm-4.5-air while preserving the possibility of using more capable proprietary models for complex reasoning. This avoids vendor lock-in and improves long-term adaptability.

Thinking Mode is implemented as a request-time prompt modification strategy. When enabled, the system appends reasoning-oriented guidance so that the model structures its answer more carefully. This does not change the base model, but it changes the inference context in a predictable way.

In the context of streaming response flow, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how streaming response flow contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The response governance layer is important because the underlying model may produce references to its provider, training background, or system behavior that are not aligned with the intended assistant identity. ENGGBOT therefore applies a controlled interception step before presenting the final answer.

This step improves user trust and gives the application a stable product identity. The same layer can also be extended later for safety checks, institution-specific answer policies, and audit logging without changing the user interface.

5.5 SPEECH-TO-TEXT FLOW

The implementation is divided across the main application, AI user interface, API routes, speech modules, and data services. The frontend provides an animated landing experience and a feature-rich chat interface. The backend provides authentication, model interaction, transcription endpoints, and persistence support.

OpenRouter integration gives ENGGBOT a model-agnostic foundation. The application can route requests to cost-effective open-source models such as glm-4.5-air while preserving the possibility of using more capable proprietary models for complex reasoning. This avoids vendor lock-in and improves long-term adaptability.

Thinking Mode is implemented as a request-time prompt modification strategy. When enabled, the system appends reasoning-oriented guidance so that the model structures its answer more carefully. This does not change the base model, but it changes the inference context in a predictable way.

In the context of speech-to-text flow, the project emphasizes practical engineering trade-offs. The design avoids binding the platform to a single model or speech provider and instead treats each external service as a replaceable component behind a stable application-level workflow.

This section also highlights how speech-to-text flow contributes to the final system. The goal is not only to make the assistant answer questions, but also to make the platform resilient, maintainable, and suitable for future academic and production use.

The speech layer is treated as an operational subsystem rather than as an optional accessory. Audio input must pass through format validation, transcription selection, response assembly, and fallback handling before it can be used reliably in a conversational workflow.

This design is useful in practical deployment because microphone quality, network availability, model latency, and server readiness can vary across sessions. By documenting these conditions, the project shows how speech support can remain usable even when one transcription service is unavailable.

Chapter 5
Conclusion And Future Work

ENGGBOT demonstrates how a modern conversational AI platform can be engineered beyond the limitations of a simple API wrapper. The system combines model-agnostic routing, dynamic prompt control, response interception, and multi-modal speech processing into a single coherent architecture.

The project is significant because it treats model providers as replaceable back-end services instead of fixed application dependencies. This allows the platform to balance cost, reasoning quality, reliability, and future extensibility.

Future work can include policy-based model selection, stronger observability, retrieval-augmented course-material ingestion, automated evaluation of generated responses, improved speech failover, and institutional deployment hardening.

REFERENCES

[1] OpenRouter Documentation. Model routing, provider abstraction, and chat completion API concepts.
[2] React Documentation. Component-based user interface development and state-driven rendering.
[3] Next.js Documentation. Application routing, server routes, and deployment workflow.
[4] NVIDIA Riva Documentation. Speech AI services, automatic speech recognition, and gRPC-based deployment.
[5] OpenAI Whisper Technical Documentation. Speech recognition model behavior and transcription workflows.
[6] Google OAuth Documentation. Authentication, authorization, and identity provider integration.
[7] Supabase Documentation. PostgreSQL-backed application data, authentication support, and hosted database usage.
[8] Wei, J. et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.
[9] Brown, T. et al. Language Models are Few-Shot Learners.
[10] Radford, A. et al. Robust Speech Recognition via Large-Scale Weak Supervision.
[11] Vaswani, A. et al. Attention Is All You Need.
[12] NVIDIA Developer Documentation. Riva automatic speech recognition deployment and client integration notes.
[13] Vercel Documentation. Next.js application hosting, environment variables, and serverless route behavior.
[14] PostgreSQL Documentation. Relational data design, indexing, and hosted database operation concepts.
[15] OAuth 2.0 Authorization Framework. Authentication delegation and secure access-token exchange.
[16] MDN Web Docs. Fetch API, streaming responses, and browser-based media capture concepts.

APPENDICES

```
# AI_UI/src/lib/ai/openrouter-client.ts



export const AVAILABLE_MODELS = {
   "gpt-oss": "google/gemini-2.0-flash-lite-preview-02-05:free",
};


interface OpenRouterClientOptions {
   apiKey?: string;
   defaultModel?: string;
}


interface GenerateOptions {
   prompt: string;
   model?: string;
   temperature?: number;
   max_tokens?: number;
   thinking_mode?: boolean;
   messages?: Array<{ role: string, content: string }>;
   stream?: boolean;
}


export class OpenRouterClient {
   private apiKey: string;
   private apiUrl: string;
   private defaultModel: string;
   private headers: Record<string, string>;

   constructor(options?: OpenRouterClientOptions) {
      this.apiKey = options?.apiKey?.trim() || "";
      this.apiUrl = "https://openrouter.ai/api/v1/chat/completions";
```

```
                    this.defaultModel = options?.defaultModel ||
AVAILABLE_MODELS["gpt-oss"];


            const refererUrl = typeof process !== 'undefined' &&
process.env.VERCEL_URL
        ? `https://${process.env.VERCEL_URL}`
                        : (typeof process !== 'undefined' &&
process.env.NEXT_PUBLIC_SITE_URL
            ? process.env.NEXT_PUBLIC_SITE_URL
            : "https://enggbot.me");

      this.headers = {
        "Authorization": `Bearer ${this.apiKey}`,
        "Content-Type": "application/json",
        "HTTP-Referer": refererUrl,
        "X-Title": "EnggBot"
      };

      if (!this.apiKey) {
          console.warn("OpenRouterClient initialized without an API
key.");
      }

          console.log(`Initialized OpenRouter client with model: $
{this.defaultModel}`);
  }


  async generate(options: GenerateOptions): Promise<string> {
    try {
      const { prompt, model, temperature = 0.7, max_tokens = 1024,
thinking_mode =
false, messages, stream = false } = options;


      const modelName = model || this.defaultModel;


      let messagePayload;
```

```
        if (messages && messages.length > 0) {

            messagePayload = messages;


            if (thinking_mode) {
                const lastMsg = messagePayload[messagePayload.length
- 1];
                if (lastMsg.role === 'user') {
                    const thinkingPrompt = prompt.includes("?")
                        ? " Please think step by step and show your reasoning
process."
                        : " Please think step by step and explain your thought
process.";


                        messagePayload = [...messagePayload.slice(0, -1),
{ ...lastMsg,
content: lastMsg.content + thinkingPrompt }];
                }
            }
        } else {

            let actualPrompt = prompt;
            if (thinking_mode) {
                if (prompt.includes("?")) {
                    actualPrompt = prompt + " Please think step by step and
show your
reasoning process.";
                } else {
                    actualPrompt = prompt + " Please think step by step and
explain your
 thought process.";
                }
            }


                        messagePayload = [{ "role": "user", "content":
actualPrompt }];
```

```
        }

        const payload = {
            "model": modelName,
            "messages": messagePayload,
            "temperature": temperature,
            "max_tokens": max_tokens,
            "stream": stream
        };

        const controller = new AbortController();
        const timeoutId = setTimeout(() => controller.abort(), 60000);

        try {
            const response = await fetch(this.apiUrl, {
                method: 'POST',
                headers: this.headers,
                body: JSON.stringify(payload),
                signal: controller.signal
            });

            clearTimeout(timeoutId);

            if (!response.ok) {
                const errorText = await response.text().catch(() => "No
error details
available");
                console.warn(`OpenRouter API error for model ${
{modelName}
(${response.status}): ${errorText}`);
                throw new Error(`API Error: ${response.status} ${
{response.statusText} -
${errorText.substring(0, 200)}`);
            }

            const result = await response.json();
            if (result.choices && result.choices.length > 0) {
                return result.choices[0].message.content;
            } else {
```

```
                return "No content returned from the API.";
            }
        } catch (err) {
            clearTimeout(timeoutId);
            throw err;
        }
    } catch (error) {
        console.error("Error generating AI response:", error);
            return `Error: ${error instanceof Error ? error.message :
String(error)}`;
    }
  }


            async generateStream(options: GenerateOptions):
Promise<ReadableStream> {
     const { prompt, model, temperature = 0.7, max_tokens = 1024,
thinking_mode = false,
messages } = options;


    const modelName = model || this.defaultModel;


    let messagePayload;
    if (messages && messages.length > 0) {

        messagePayload = messages;


        if (thinking_mode) {
            const lastMsg = messagePayload[messagePayload.length -
1];
            if (lastMsg.role === 'user') {
                const thinkingPrompt = prompt.includes("?")
                    ? " Please think step by step and show your reasoning
process."
                    : " Please think step by step and explain your thought
process.";
```

```
                    messagePayload = [...messagePayload.slice(0, -1),
{ ...lastMsg, content:
 lastMsg.content + thinkingPrompt }];
            }
        }
    } else {

        let actualPrompt = prompt;
        if (thinking_mode) {
            if (prompt.includes("?")) {
                actualPrompt = prompt + " Please think step by step and
show your
reasoning process.";
            } else {
                actualPrompt = prompt + " Please think step by step and
explain your
thought process.";
            }
        }


                    messagePayload = [{ "role": "user", "content":
actualPrompt }];
    }

    const payload = {
        "model": modelName,
        "messages": messagePayload,
        "temperature": temperature,
        "max_tokens": max_tokens,
        "stream": true
    };

    const controller = new AbortController();


    try {
        const response = await fetch(this.apiUrl, {
```

```
            method: 'POST',
            headers: this.headers,
            body: JSON.stringify(payload),
            signal: controller.signal
          });

          if (!response.ok) {
            const errorText = await response.text().catch(() => "No error
details
available");
                        console.warn(`OpenRouter API error for model $
{modelName}
(${response.status}): ${errorText}`);
                        throw new Error(`API Error: ${response.status} $
{response.statusText} -
${errorText.substring(0, 200)}`);
          }


          return response.body!;
        } catch (err) {
          throw err;
        }
    }


    switchModel(modelKey: string): boolean {
        if (modelKey in AVAILABLE_MODELS) {
            this.defaultModel = AVAILABLE_MODELS[modelKey as
keyof typeof AVAILABLE_MODELS];
          console.log(`Switched to model: ${this.defaultModel}`);
          return true;
        } else {
            console.log(`Model key '${modelKey}' not found. Available
keys:
${Object.keys(AVAILABLE_MODELS)}`);
          return false;
        }
    }
```

```
    listModels(): Record<string, string> {
        return AVAILABLE_MODELS;
    }
}
```

AI_UI/src/lib/ai/response-middleware.ts

```
export const BOT_CONFIG = {
  NAME: "ENGGBOT",
    PERSONALITY: "helpful, friendly, and enthusiastic engineering assistant",
  VERSION: "1.0",
  EMOJI: {
    DEFAULT: "🛠️",
    THINKING: "💡",
    SUCCESS: "✅",
    ERROR: "⚠️",
    CODE: "💻",
    IDEA: "💡"
  }
};

export const EXACT_IDENTITY_REPLY = "I'm ENGGBOT, an AI assistant";

const IDENTITY_PATTERNS = [
  /who (are|created|made|developed|built|designed) you/i,
  /what (are|is) you/i,
  /your name/i,
  /which (model|ai|assistant|llm)/i,
  /what model/i,
  /what (version|language model)/i,
```

```
  /tell me about (yourself|you)/i,
  /made you/i,
  /developed you/i,
  /what's your identity/i,
  /deepseek/i,
  /z\.?ai/i,
  /glm[-\s]?4(\.\d+)?/i,
  /zhipu/i,
  /who (is|are|runs|owns|created|made|built|designed) (your|the)
(provider|company|creator|developer|model|llm|foundation|base)/i,
  /what (is|are) (your|the)
(provider|company|creator|developer|model|llm|foundation|base)/i,
       /what (company|organization) (made|created|developed|built|
designed|runs|owns) you/i,
                              /where (are|were) you
(made|created|developed|built|designed|from)/i,
   /what (country|nation) (are|were) you (made|created|developed|built|
designed) in/i,
  /who (provides|is providing) (your|the) (ai|model|llm)/i,
  /which (company|organization|country) (are you|you are) from/i,
  /are you (made|developed|created|built|designed) by/i
];

// Additional sensitive keywords that should trigger identity response
const SENSITIVE_KEYWORDS = [
  'deepseek', 'deep seek', 'z.ai', 'zai', 'glm', 'glm air', 'zhipu',
  'chinese', 'china', 'llm provider', 'model provider',
  'ai provider', 'provider', 'open source', 'foundation model', 'base model',
  'language model'
];

// Check if a message is asking about the AI's identity
export function isIdentityQuestion(message: string): boolean {

  const lowercaseMessage = message.toLowerCase();


   const matchesPattern = IDENTITY_PATTERNS.some(pattern =>
pattern.test(lowercaseMessage));
```

```
                const containsSensitiveKeyword =
SENSITIVE_KEYWORDS.some(keyword =>
   lowercaseMessage.includes(keyword.toLowerCase())
  );

  return matchesPattern || containsSensitiveKeyword;
}


export function generateIdentityResponse(): string {
    const { NAME, PERSONALITY, VERSION, EMOJI } =
BOT_CONFIG;


  return `${EMOJI.IDEA} I'm ${NAME} (v${VERSION}), your $
{PERSONALITY}!

I'm a custom-built engineering assistant designed specifically to help
with coding,
technical problems, and engineering challenges. I'm not based on any
existing public AI
model or service.

${EMOJI.CODE} My focus is on providing practical solutions to
engineering problems,
explaining technical concepts clearly, and helping you with your code.
I can assist with
many programming languages and development tasks.

${EMOJI.DEFAULT} I was built from the ground up as ${NAME} -
that's my complete identity!
I'm here to assist you with whatever engineering challenges you're
facing today.`;
}


export function generateProviderResponse(): string {
  const { NAME, EMOJI } = BOT_CONFIG;
```

return `${EMOJI.IDEA} ${NAME} is a purpose-built engineering assistant created by a
specialized team of developers.

I'm designed from the ground up specifically as ${NAME} - not based on any public large
language model. My purpose is to focus exclusively on engineering tasks, coding assistance,
and technical problem-solving.

${EMOJI.CODE} I'm maintained by a dedicated engineering team that continually improves my
abilities to better assist with your technical questions and coding challenges.

Is there a specific engineering task or coding challenge I can help you with today?`;
}

export function generateMarkdownSystemPrompt(): string {
    return `You are ${BOT_CONFIG.NAME}, a ${BOT_CONFIG.PERSONALITY}.

When responding, use Markdown formatting to enhance readability:

1. Use **bold** for emphasis and important points
2. Use *italics* for definitions or to highlight terms
3. Use \`code\` for inline code snippets
4. Use code blocks with language specification for multi-line code:
   \`\`\`javascript
   // Example code
   function hello() {
     console.log("Hello world");
   }
   \`\`\`
5. Use bullet points or numbered lists for sequential items
6. Use ## and ### for section headers
7. Use > for quotes or important notes

8. Use --- for horizontal separators between sections
9. Use [text](url) for links

Always maintain this formatting style to ensure your responses are clear, well-structured,
and easy to read.`;
}

export function processAIResponse(response: string, userMessage: string): string {

 if (isIdentityQuestion(userMessage)) {

  return EXACT_IDENTITY_REPLY;
 }

 return sanitizeResponse(response);
}

function sanitizeResponse(text: string): string {
 const { NAME, EMOJI } = BOT_CONFIG;

 let sanitized = text

  .replace(/<\uFF5Cbegin▁of▁sentence\uFF5C>/g, '')
  .replace(/<\|begin▁of▁sentence\|>/g, '')
  .replace(/<\|begin_of_sentence\|>/gi, '')

  .replace(/DeepSeek/gi, NAME)
  .replace(/Deep Seek/gi, NAME)
  .replace(/Deepseek AI/gi, NAME)
  .replace(/Z\.?AI/gi, NAME)
  .replace(/ZAI/gi, NAME)
  .replace(/Zhipu/gi, NAME)
  .replace(/GLM[-\s]?4(?:\.\d+)?(?:\s*Air)?/gi, NAME)
  .replace(/GLM[-\s]?4\.5v/gi, NAME)
  .replace(/OpenRouter/gi, NAME)

```
    .replace(/Open Router/gi, NAME)
    .replace(/OpenAI/gi, NAME)
    .replace(/GPT-?\w*/gi, NAME)
    .replace(/Anthropic/gi, NAME)
    .replace(/Claude/gi, NAME)
    .replace(/Mistral/gi, NAME)
    .replace(/Google/gi, NAME)
    .replace(/Gemini/gi, NAME)
    .replace(/Chutes( AI)?/gi, NAME)
    .replace(/NVIDIA/gi, NAME)
    .replace(/Riva/gi, NAME)
    .replace(/Llama/gi, NAME)
    .replace(/Meta/gi, NAME)
    .replace(/Groq/gi, NAME)
    .replace(/Cohere/gi, NAME)
    .replace(/Perplexity/gi, NAME)
    .replace(/xAI/gi, NAME)
    .replace(/OpenPipe/gi, NAME)
    .replace(/Vercel( AI)?/gi, NAME)
    .replace(/LangChain/gi, NAME)

    .replace(/based in China/gi, "")
    .replace(/from China/gi, "")
    .replace(/Chinese (AI|company|model|assistant)/gi, `${NAME}`)
    .replace(/I('m| am) (a|an)
(DeepSeek|Deepseek|Chinese|AI|language|LLM|GLM|Z\.?AI|
Zhipu).*(model|assistant)/gi, `I'm
${NAME}`)
        .replace(/My (provider|creator|developer|company) is/gi, `I'm $
{NAME}, and`)
    .replace(/I('m| am) powered by/gi, `I'm ${NAME}, `)
    .replace(/I('m| am) developed by/gi, `I'm ${NAME}, `)
    .replace(/I('m| am) created by/gi, `I'm ${NAME}, `)
    .replace(/I('m| am) made by/gi, `I'm ${NAME}, `);



  return sanitized;
```

```
}
```

AI_UI/src/app/api/chat/route.ts

```
import { NextResponse } from 'next/server';
import { v4 as uuidv4 } from 'uuid';
import { AVAILABLE_MODELS, OpenRouterClient } from '@/lib/ai/openrouter-client';
import { processAIResponse, BOT_CONFIG, generateMarkdownSystemPrompt, isIdentityQuestion, EXACT_IDENTITY_REPLY } from '@/lib/ai/response-middleware';
import { isClientInitialized, initializeAIClient } from '@/lib/ai/preload-client';
import crypto from 'crypto';

export const runtime = 'nodejs';


export interface ChatMessage {
  id: string;
  content: string;
  isUser: boolean;
  timestamp: string;
}


function formatConversationHistory(history: ChatMessage[]): Array<{ role: string, content:
string }> {
  if (!history || history.length === 0) return [];

  return history.map(msg => {
    const role = msg.isUser ? 'user' : 'assistant';
    return { role, content: msg.content };
  });
}


export async function POST(request: Request) {
```

```
 try {
  const {
   message,
   hasAttachments = false,
   model = "z-ai/glm-4.5-air:free",
   thinkingMode = true,
   conversationHistory = []
  } = await request.json();

  if (!message || typeof message !== 'string') {
   return NextResponse.json(
    { error: 'Message is required and must be a string' },
    { status: 400 }
   );
  }


  if (isIdentityQuestion(message)) {
   const chatMessage: ChatMessage = {
    id: crypto.randomUUID(),
    content: EXACT_IDENTITY_REPLY,
    isUser: false,
    timestamp: new Date().toISOString()
   };
   return NextResponse.json({ message: chatMessage });
  }


  if (!isClientInitialized()) {
   await initializeAIClient();
  }


  const apiKey = process.env.OPENROUTER_API_KEY;
  if (!apiKey) {
     console.warn("OPENROUTER_API_KEY is not set. Configure
this env var in production.");
  }
  const openRouterClient = new OpenRouterClient({ apiKey });
```

```
                                const formattedMessages = formatConversationHistory(conversationHistory);


   const systemMessage = {
    role: 'system',
    content: generateMarkdownSystemPrompt()
   };


          const messages = formattedMessages.length > 0 && formattedMessages[0].role === 'system'
?
    formattedMessages :
    [systemMessage, ...formattedMessages];


   messages.push({
    role: 'user',
    content: hasAttachments
          ? `${message} (The user has also provided some files or attachments with this
message)`
      : message
   });


   const modelName = AVAILABLE_MODELS["gpt-oss"];

   try {

    const response = await openRouterClient.generate({
     prompt: message,
     model: modelName,
     temperature: 0.7,
     max_tokens: 1024,
     thinking_mode: thinkingMode,
     messages: messages
```

```
      });


            const processedResponse = processAIResponse(response,
message);


      const chatMessage: ChatMessage = {
        id: crypto.randomUUID(),
        content: processedResponse,
        isUser: false,
        timestamp: new Date().toISOString()
      };

      return NextResponse.json({ message: chatMessage });

    } catch (error) {
      console.error("Error generating AI response:", error);


      const errorMessage: ChatMessage = {
        id: crypto.randomUUID(),
           content: "I apologize, but I'm having trouble processing your
request right now.
Please try again.",
        isUser: false,
        timestamp: new Date().toISOString()
      };

      return NextResponse.json({ message: errorMessage });
    }
  } catch (error) {
    console.error('Error processing chat message:', error);
    return NextResponse.json(
      { error: 'Failed to process your message' },
      { status: 500 }
    );
  }
}
```

```
# AI_UI/src/app/api/chat/stream/route.ts

import { NextResponse } from 'next/server';
import { AVAILABLE_MODELS, OpenRouterClient } from '@/lib/ai/openrouter-client';
import { processAIResponse, BOT_CONFIG, generateMarkdownSystemPrompt, isIdentityQuestion, EXACT_IDENTITY_REPLY } from '@/lib/ai/response-middleware';
import { ENGINEERING_SYSTEM_PROMPT } from '@/lib/prompts/engineering-prompt';
import { getPredefinedAnswer } from '@/lib/ai/predefined-answers';

export const runtime = 'nodejs';

function processOpenRouterStream(
  stream: ReadableStream,
  userMessage: string,
  encoder = new TextEncoder(),
  decoder = new TextDecoder()
): ReadableStream {
  const reader = stream.getReader();

  return new ReadableStream({
    async start(controller) {
      let isClosed = false;

      const safeEnqueue = (chunk: Uint8Array) => {
        try {
          if (!isClosed) {
            controller.enqueue(chunk);
          }
        } catch {
          isClosed = true;
        }
      };

      const safeClose = () => {
```

```
  try {
    if (!isClosed) {
      isClosed = true;
      controller.close();
    }
  } catch {
    isClosed = true;
  }
};

try {
  let buffer = '';
  let doneSent = false;

  while (true) {
    const { done, value } = await reader.read();
    if (done) {
      break;
    }
    buffer += decoder.decode(value, { stream: true });

    const lines = buffer.split('\n');
    buffer = lines.pop() || '';

    for (const line of lines) {
      if (line.trim() === '') continue;
      if (!line.startsWith('data:')) continue;

      try {
        const jsonData = line.slice(5).trim();
        if (!jsonData) continue;

        if (jsonData === "[DONE]") {
          safeEnqueue(encoder.encode(`data: [DONE]\n\n`));
          doneSent = true;
          continue;
        }
        const data = JSON.parse(jsonData);
```

```
            if (data.choices && data.choices[0]?.delta?.content) {
              const content = data.choices[0].delta.content;
                    const processedContent = processAIResponse(content,
userMessage);
                          const event = `data: ${JSON.stringify({ text:
processedContent })}\n\n`;
              safeEnqueue(encoder.encode(event));
             }
            } catch (e) {
             console.error("Error processing stream line:", e, "Line:", line);
             continue;
            }
           }
          }

          // Only send [DONE] if it wasn't already sent from the upstream
data
          if (!doneSent) {
           safeEnqueue(encoder.encode(`data: [DONE]\n\n`));
          }
         } catch (error) {
          console.error("Stream processing error:", error);
          if (!isClosed) {
           isClosed = true;
           controller.error(error);
          }
         } finally {
          safeClose();
         }
        },

        async cancel() {
         await reader.cancel();
        }
       });
      }

      function formatConversationHistory(history: any[]): Array<{ role:
```

```
string, content: string }>
 {
  if (!history || history.length === 0) return [];

  return history.map(msg => {
    const role = msg.isUser ? 'user' : 'assistant';
    return { role, content: msg.content };
  });
}
export async function POST(request: Request) {
  try {
    const {
      message,
      rawMessage,
      hasAttachments = false,
      model = "z-ai/glm-4.5-air:free",
      thinkingMode = true,
      engineeringMode = false,
      conversationHistory = []
    } = await request.json();

    if (!message || typeof message !== 'string') {
      return NextResponse.json(
        { error: 'Message is required and must be a string' },
        { status: 400 }
      );
    }

       const identityProbeText = typeof rawMessage === 'string' &&
rawMessage.trim().length > 0
 ? rawMessage : message;

    // ── Predefined answers – stream in chunks to feel natural (3-5s)
──

    const predefined = getPredefinedAnswer(identityProbeText);
    if (predefined) {
      const encoder = new TextEncoder();

     // Split answer into small word-based chunks for realistic streaming
```

```
    const words = predefined.split(/(\s+)/); // keep whitespace as separate tokens
    const CHUNK_SIZE = 6; // words per chunk
    const chunks: string[] = [];
    for (let i = 0; i < words.length; i += CHUNK_SIZE) {
      chunks.push(words.slice(i, i + CHUNK_SIZE).join(''));
    }
    // Target ~4 seconds total; compute delay per chunk
    const totalMs = 6000;
        const delayMs = Math.max(15, Math.floor(totalMs / chunks.length));

    const stream = new ReadableStream({
      async start(controller) {
        try {
          for (const chunk of chunks) {
            const event = `data: ${JSON.stringify({ text: chunk })}\n\n`;
            controller.enqueue(encoder.encode(event));
                await new Promise(resolve => setTimeout(resolve, delayMs));
          }
          controller.enqueue(encoder.encode(`data: [DONE]\n\n`));
          controller.close();
        } catch {
          controller.close();
        }
      }
    });

    return new Response(stream, {
      headers: {
        'Content-Type': 'text/event-stream',
        'Cache-Control': 'no-cache, no-transform',
        'Connection': 'keep-alive',
        'Transfer-Encoding': 'chunked',
        'X-Accel-Buffering': 'no'
      }
    });
```

```
    }

    if (isIdentityQuestion(identityProbeText)) {
      const encoder = new TextEncoder();
      const stream = new ReadableStream({
        start(controller) {
                        const event = `data: ${JSON.stringify({ text:
EXACT_IDENTITY_REPLY })}\n\n`;
          controller.enqueue(encoder.encode(event));
          const doneEvent = `data: [DONE]\n\n`;
          controller.enqueue(encoder.encode(doneEvent));
          controller.close();
        }
      });
      return new Response(stream, {
        headers: {
          'Content-Type': 'text/event-stream',
          'Cache-Control': 'no-cache, no-transform',
          'Connection': 'keep-alive',
          'Transfer-Encoding': 'chunked',
          'X-Accel-Buffering': 'no'
        }
      });
    }
    const apiKey = process.env.OPENROUTER_API_KEY;
        console.log("Debug: OPENROUTER_API_KEY present:", !!
apiKey);
       console.log("Debug: Using API Key starting with:", apiKey ?
apiKey.substring(0, 8) +
"..." : "undefined");
    if (!apiKey) {
        console.warn("OPENROUTER_API_KEY is not set. Configure
this env var in production.");
    }
    const openRouterClient = new OpenRouterClient({ apiKey });

    const modelName = AVAILABLE_MODELS["gpt-oss"];
```

```
  const messages = [];

  messages.push({
    role: 'system',
                                    content: engineeringMode ?
ENGINEERING_SYSTEM_PROMPT                                    :
generateMarkdownSystemPrompt()
  });


  if (conversationHistory && conversationHistory.length > 0) {

                                    const formattedMessages =
formatConversationHistory(conversationHistory);
    messages.push(...formattedMessages);
  }

  messages.push({
    role: 'user',
    content: hasAttachments
          ? `${message} (The user has also provided some files or
attachments with this
message)`
      : message,
  });

  try {

    const stream = await openRouterClient.generateStream({
      prompt: "",
      model: modelName,
      messages: messages,
      temperature: 0.7,
      max_tokens: 1024,
      thinking_mode: thinkingMode,
      stream: true
    });

        const processedStream = processOpenRouterStream(stream,
```

```
identityProbeText);

      return new Response(processedStream, {
        headers: {
          'Content-Type': 'text/event-stream',
          'Cache-Control': 'no-cache, no-transform',
          'Connection': 'keep-alive',
          'Transfer-Encoding': 'chunked',
          'X-Accel-Buffering': 'no'
        }
      });
    } catch (error) {
        const errorMessage = error instanceof Error ? error.message :
String(error);
            console.error("Error generating streaming response:",
errorMessage, error);
         return new Response(`data: ${JSON.stringify({ error: `Error
generating streaming
response: ${errorMessage}` })}\n\ndata: [DONE]\n\n`, {
        headers: {
          'Content-Type': 'text/event-stream',
          'Cache-Control': 'no-cache',
          'Connection': 'keep-alive'
        }
      });
    }
  } catch (error) {
    console.error('Error processing chat message:', error);
    return NextResponse.json(
      { error: 'Failed to process your message' },
      { status: 500 }
    );
  }
}


# AI_UI/src/app/api/transcribe/route.ts
```

```
export const runtime = "nodejs";

import type { NextRequest } from "next/server";

function parseBytezStream(text: string): string | null {
  try {

    const lines = text.split(/\r?\n/);
    let collected = "";
    let lastOutput: string | null = null;
    for (const line of lines) {
```